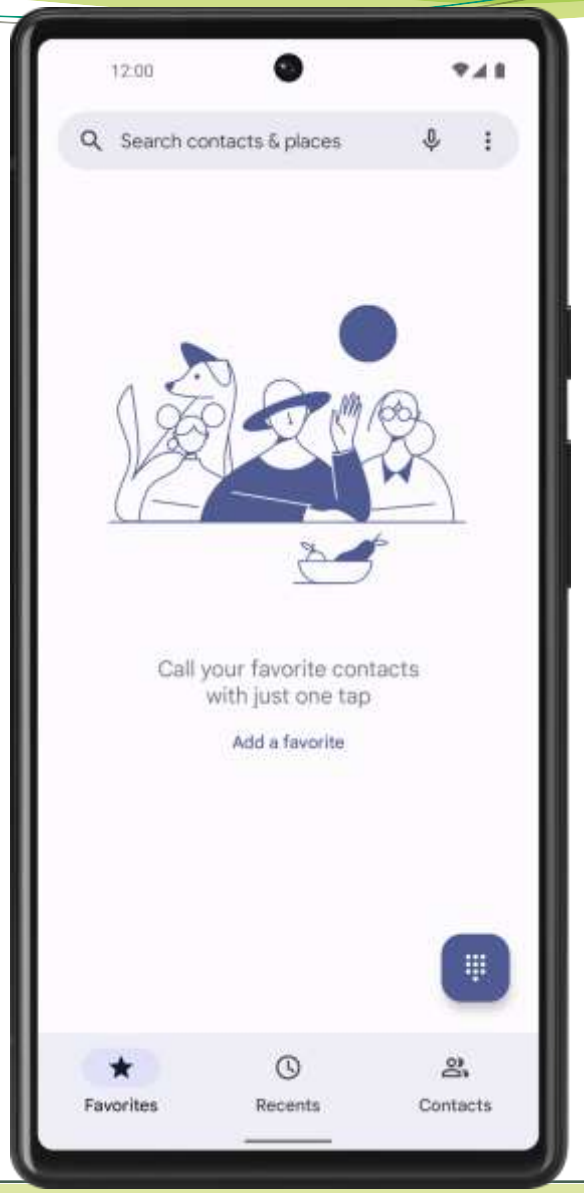
《移动终端软件开发技术》

Mobile Application Development

3. Android用户界面

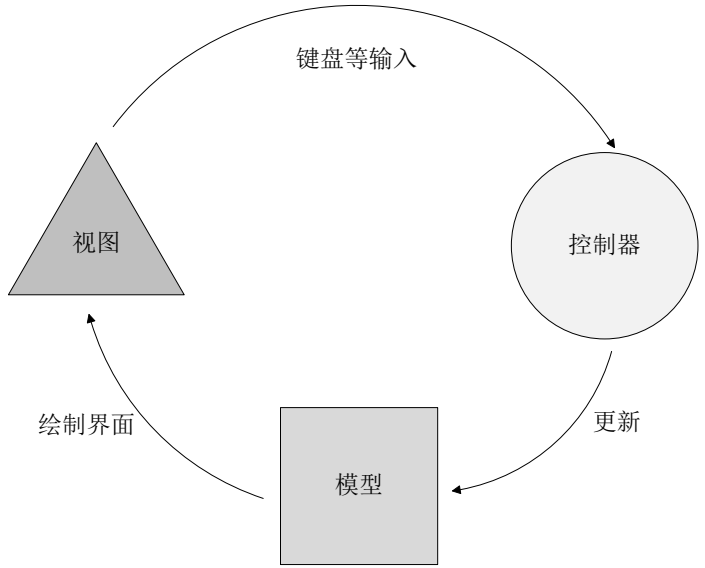
谢坤
计算机学院（国家示范性软件学院）

1 用户界面基础


12:00
Search contacts & places
Call your favorite contacts
with just one tap
Add a favorite
Favorites
Recents
Contacts

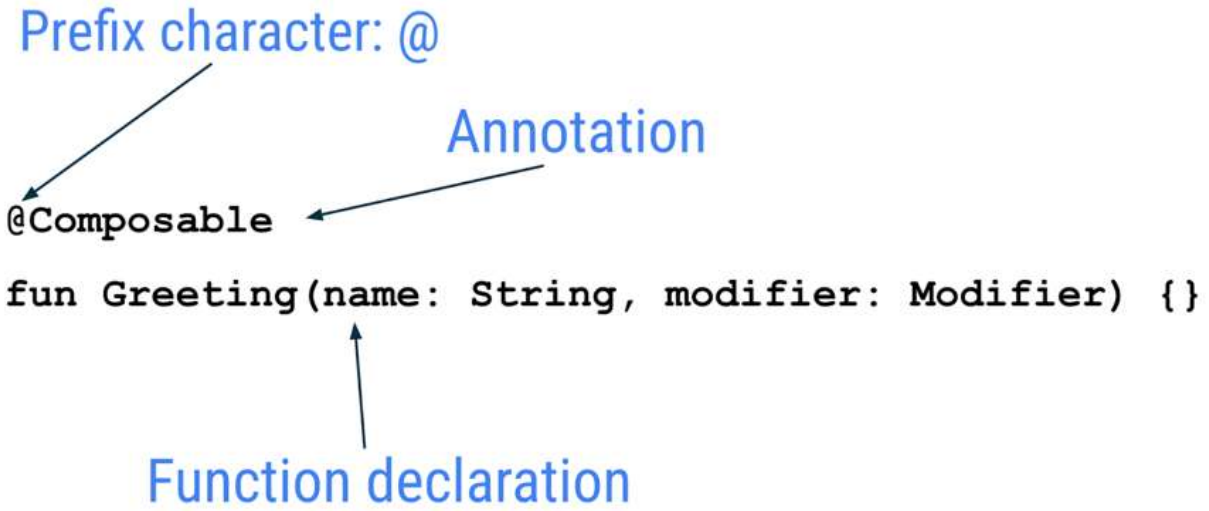
1.1 传统用户界面开发

- Android传统用户界面开发框架
  - Android传统用户界面框架采用MVC（Model-View-Controller）模型
    - 控制器（Controller）：处理用户输入
    - 视图（View）：显示用户界面和图像
    - 模型（Model）：保存数据和代码



1.3 Jetpack Compose

- Jetpack Compose
  - Android现代化UI工具包
    - 以声明式的方式构建原生Android UI
  - 界面基本构建模块：**可组合函数**
    - 描述界面中的某一部分
    - 不会返回任何内容
    - 接受一些输入并生成屏幕上显示的内容



1.3 可组合函数

- 可组合函数
  - 带有 @Composable 注解
    - 告知 Compose 编译器：此函数用于将数据转换为界面。
  - 描述应用界面的外观
  - 不关注界面的构建过程
  - 不返回任何内容
  - 接受参数
    - 描述或修改界面
  - 可组合函数名称
    - Pascal 命名法

```
1. @Composable
2. fun Greeting(name: String) {
3.     Text(text = "Hello $name!")
4. }
```

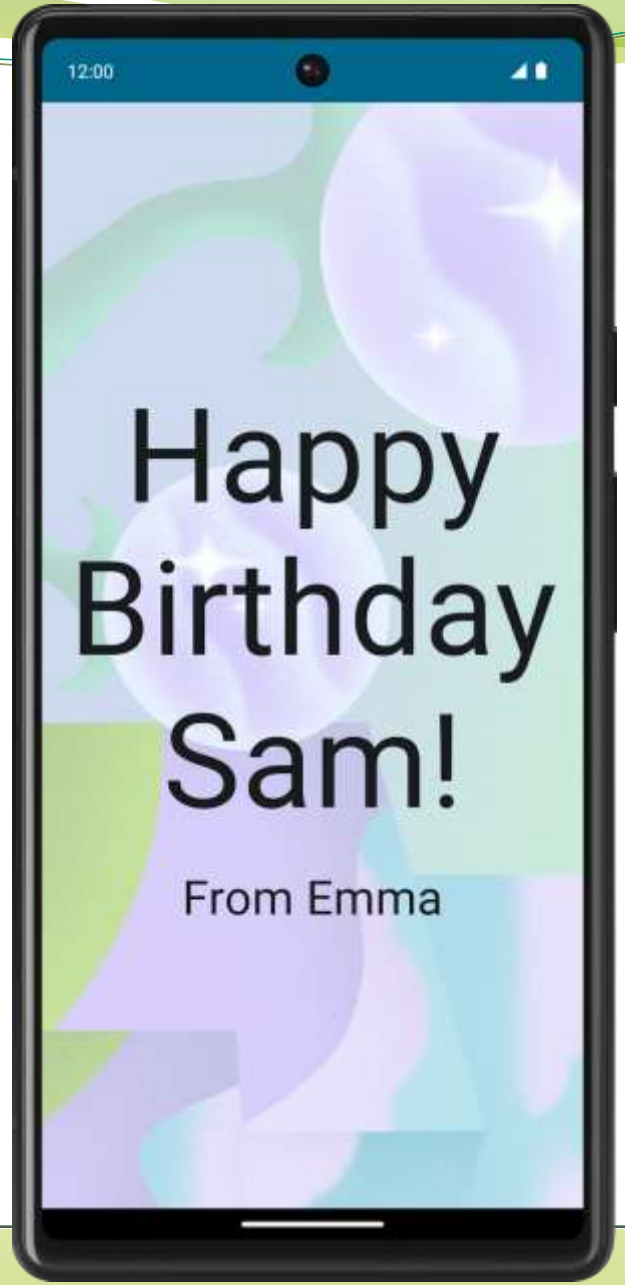
1.4 AS中的Design窗口

- 可组合函数预览
  - 添加@Preview注解
  - 为所有形参提供默认值

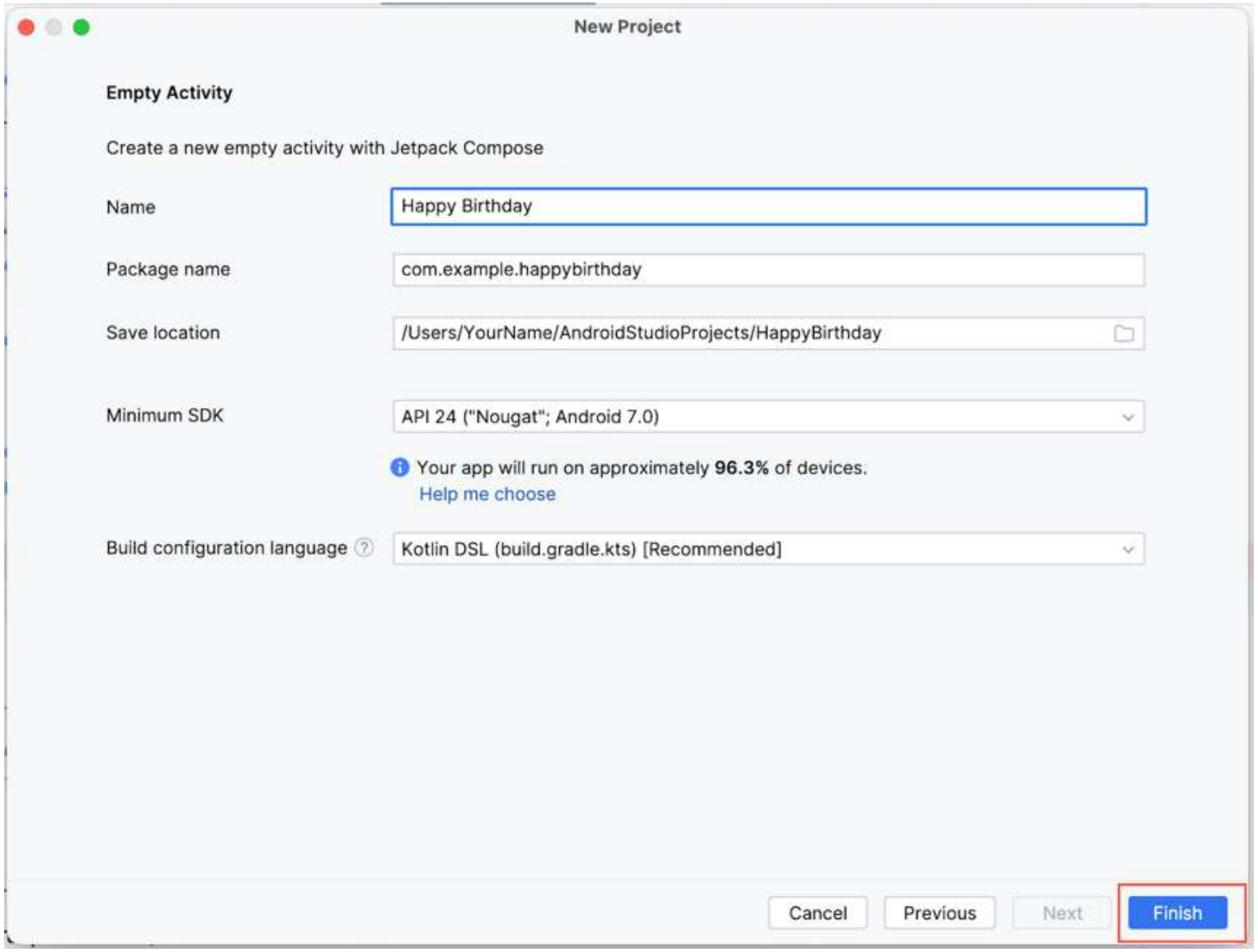
```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name!")
}
```

```
@Preview(showBackground = true)
@Composable
fun BirthdayCardPreview() {
    HappyBirthdayTheme {
        Greeting("James")
    }
}
```

2 展示信息的简单应用



2.1创建 Happy Birthday 应用



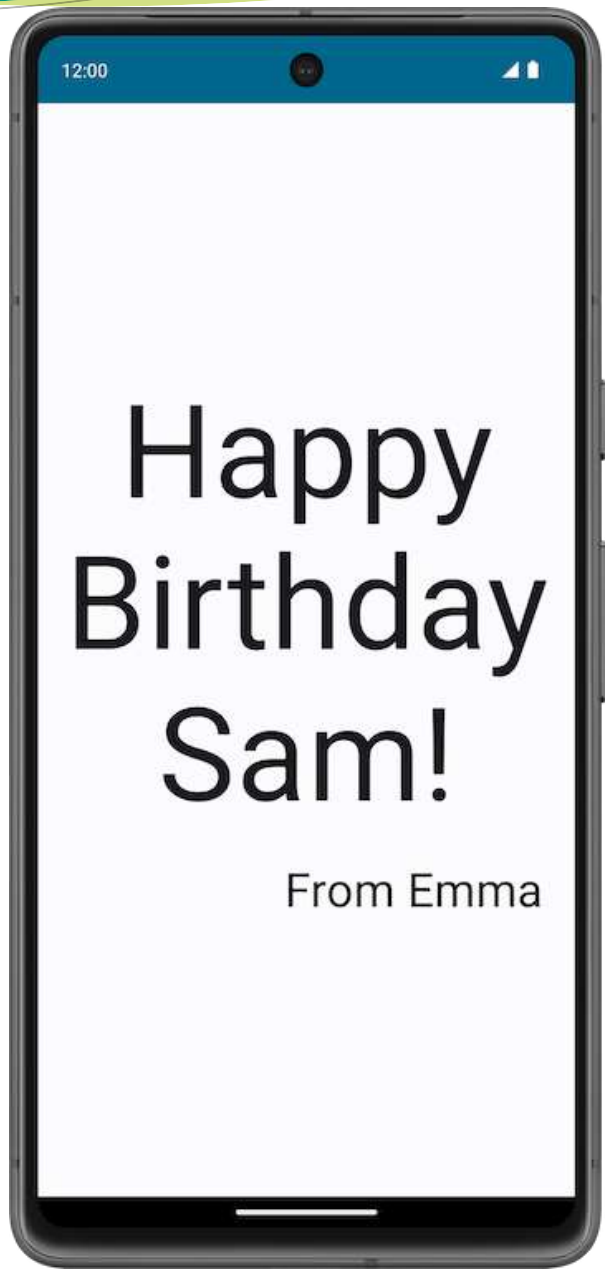
```kotlin
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MyApplicationTheme {
        Greeting( name: "Android")
    }
}
```

GreetingPreview

Hello Android!

2.2 自定义文本信息

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            HappyBirthdayTheme {
                // A surface container using the 'background' color from the theme
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                }
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun BirthdayCardPreview() {
    HappyBirthdayTheme {
    }
}
```



2.2 自定义文本信息

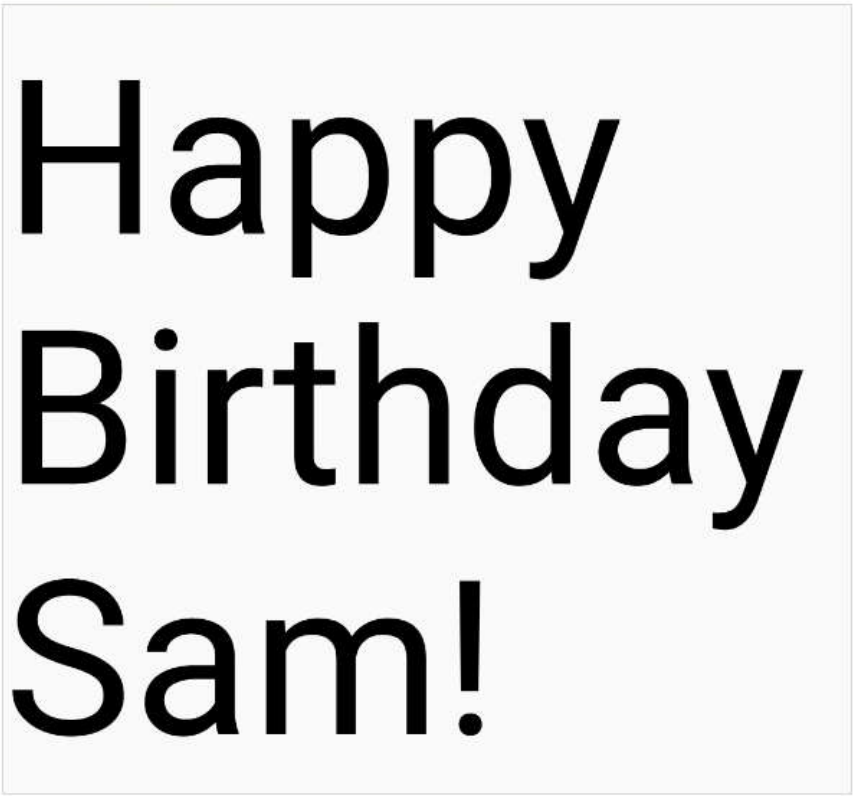
```
@Composable
fun GreetingText(message: String, modifier: Modifier = Modifier) {
    Text(
        text = message
    )
}
```

```
@Preview(showBackground = true)
@Composable
fun BirthdayCardPreview() {
    HappyBirthdayTheme {
        GreetingText(message = "Happy Birthday Sam!")
    }
}
```

2.2 自定义文本信息

```
@Composable
fun GreetingText(message: String, modifier: Modifier = Modifier) {
    Text(
        text = message,
        fontSize = 100.sp,
        lineHeight = 116.sp,
    )
}
```

BirthdayCardPreview

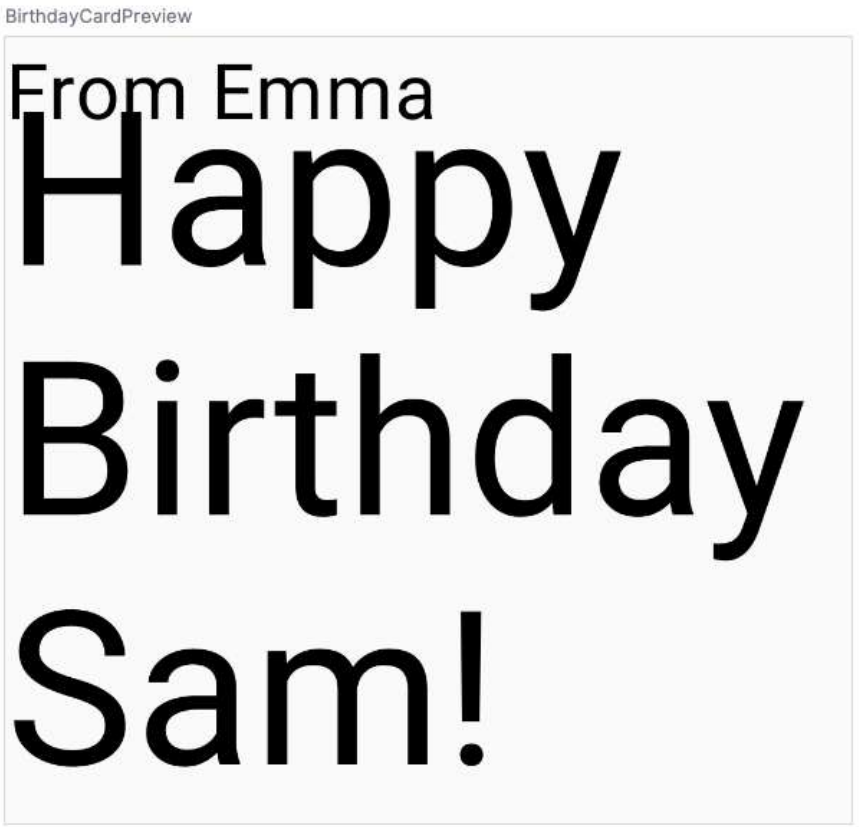


2.2 自定义文本信息

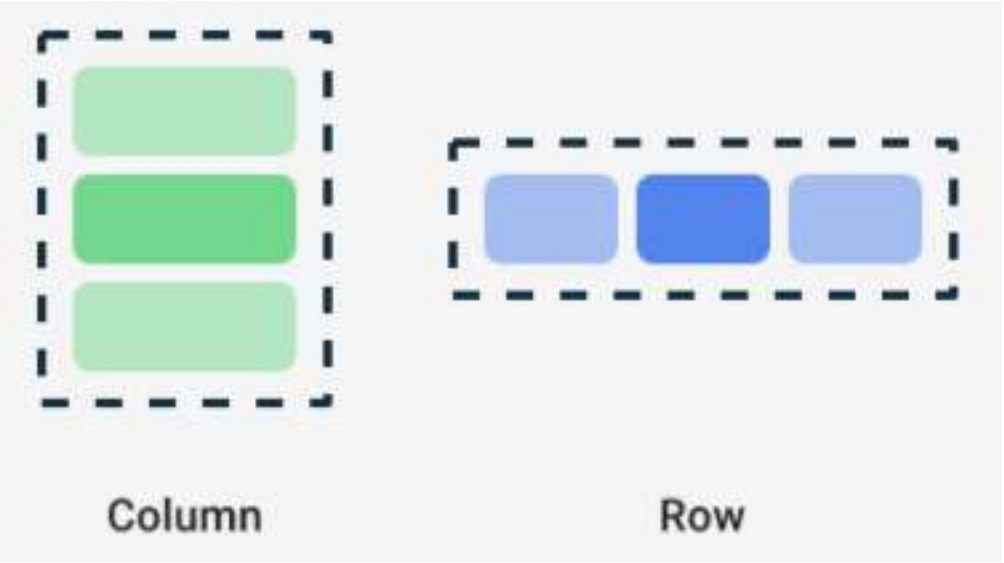
```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Text(
        // ...
    )
    Text(
        text = from
    )
}
```

```
Text(
    text = from,
    fontSize = 36.sp
)
```

```
GreetingText(message = "Happy Birthday Sam!", from = "From Emma")
```



2.2 自定义文本信息



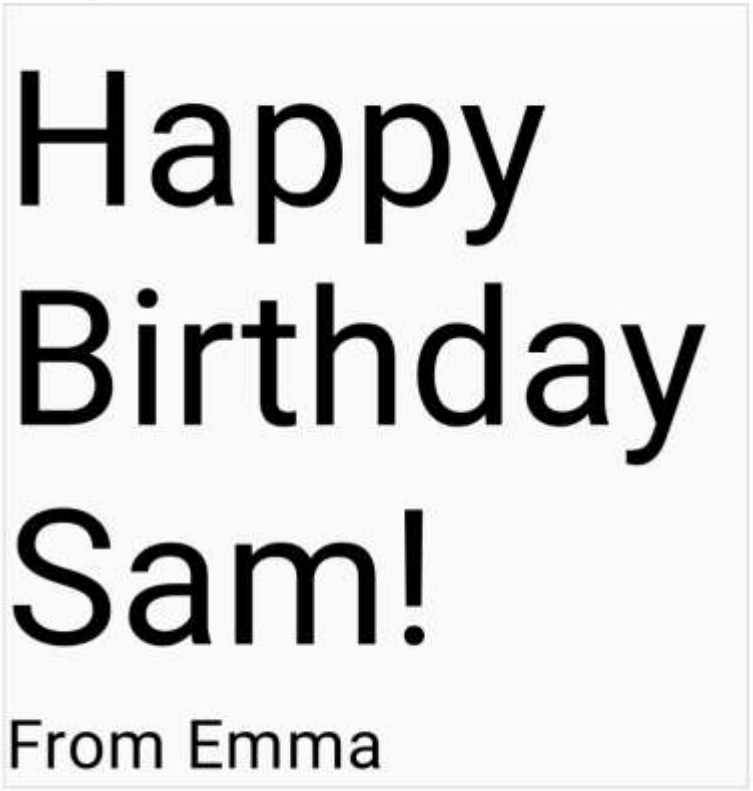

```
Row {
    Text("First Column")
    Text("Second Column")
}
```

2.2 自定义文本信息

```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Column(modifier = modifier) {
        Text(
            text = message,
            fontSize = 100.sp,
            lineHeight = 116.sp,
        )
        Text(
            text = from,
            fontSize = 36.sp
        )
    }
}
```

```
GreetingText(message = "Happy Birthday Sam!", from = "From Emma")
```

BirthdayCardPreview



2.2 自定义文本信息

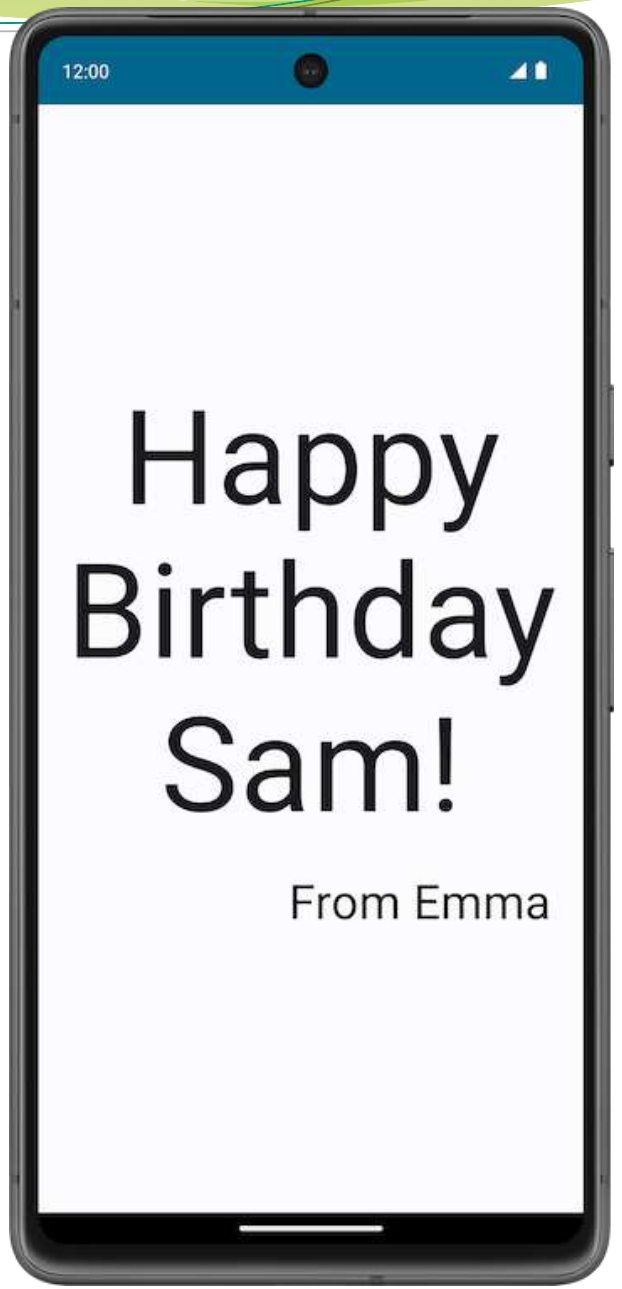
```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            HappyBirthdayTheme {
                // A surface container using the 'background' color from the theme
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    GreetingText(message = "Happy Birthday Sam!", from = "From Emma")
                }
            }
        }
    }
}
```

```
Surface(
    modifier = Modifier.fillMaxSize(),
    color = MaterialTheme.colorScheme.background
) {
    GreetingText(
        message = "Happy Birthday Sam!",
        from = "From Emma",
        modifier = Modifier.padding(8.dp)
    )
}
```

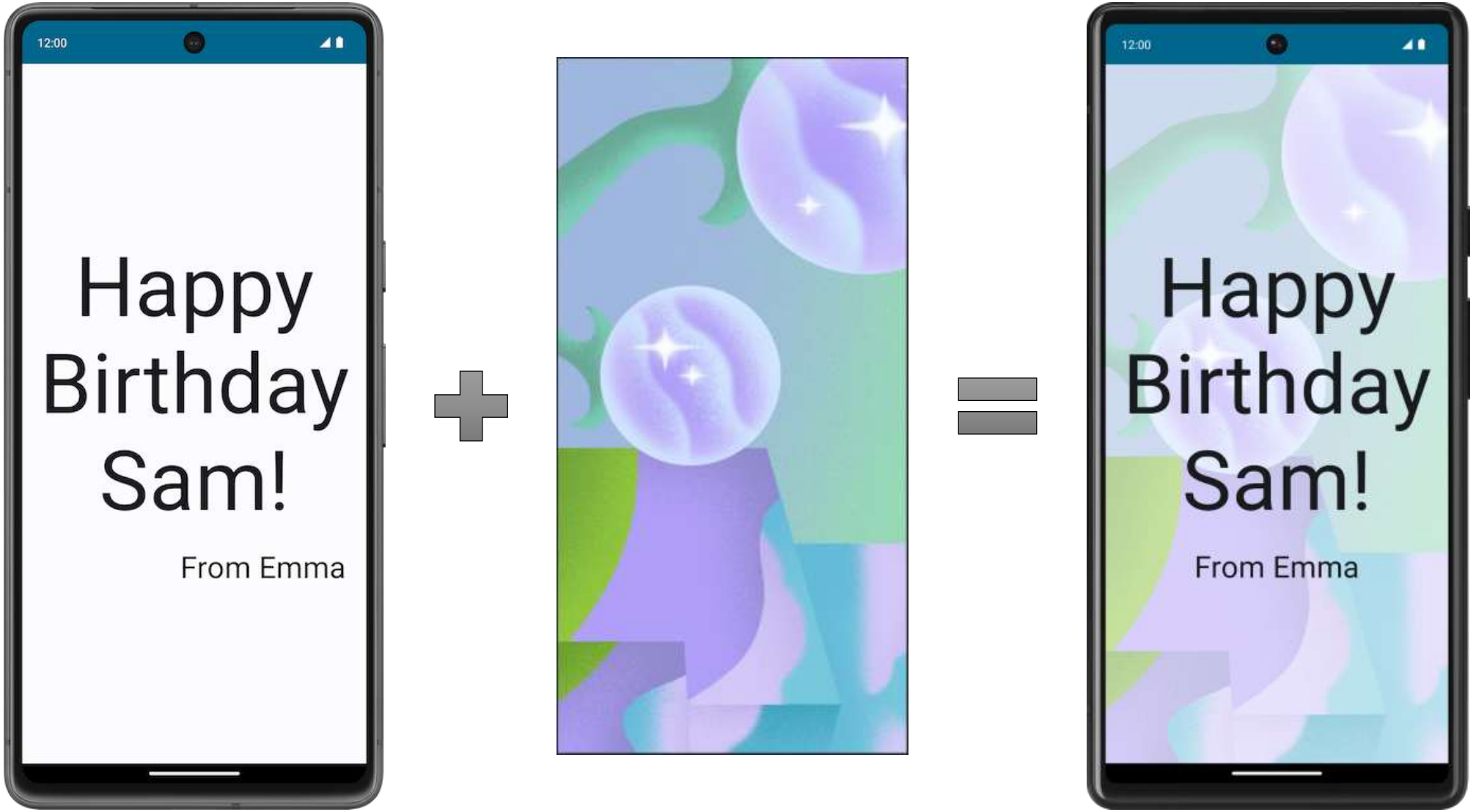
onCreate调用

```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {
    Column(
        verticalArrangement = Arrangement.Center,
        modifier = modifier
    ) {
        Text(
            text = message,
            fontSize = 100.sp,
            lineHeight = 116.sp,
            textAlign = TextAlign.Center
        )
        Text(
            text = from,
            fontSize = 36.sp,
            modifier = Modifier
                .padding(16.dp)
                .align(alignment = Alignment.End)
        )
    }
}
```

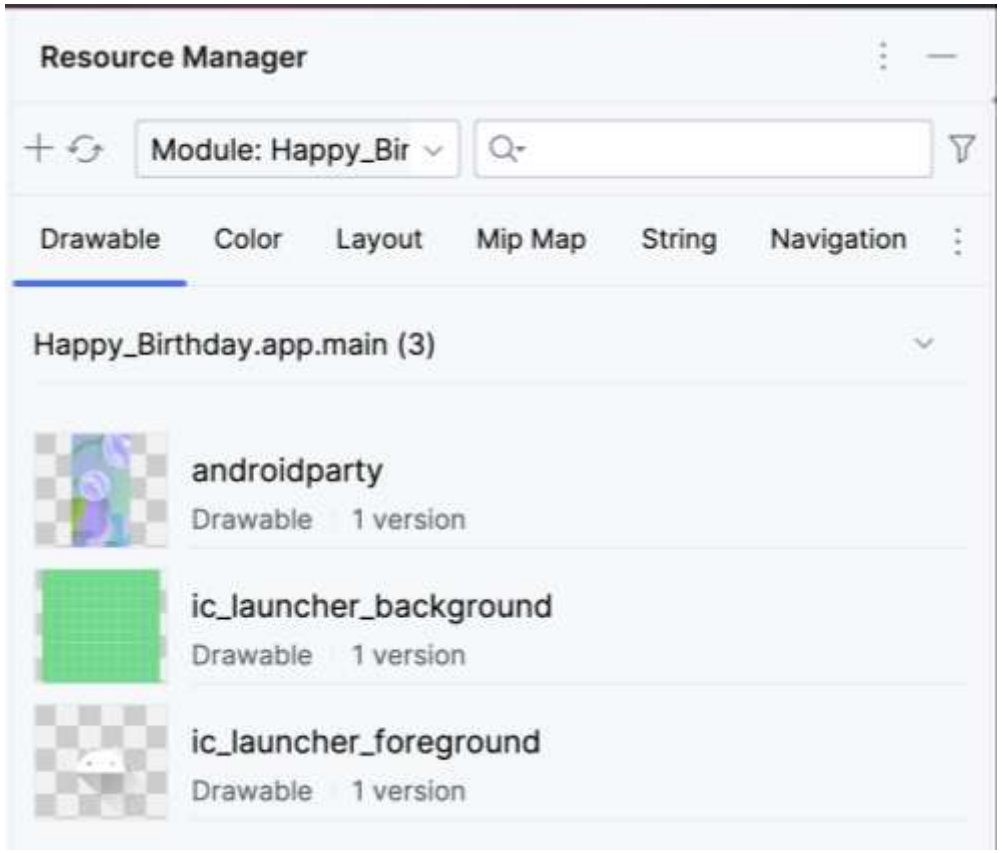
这是签名Text的modifer，不是Column传入的。

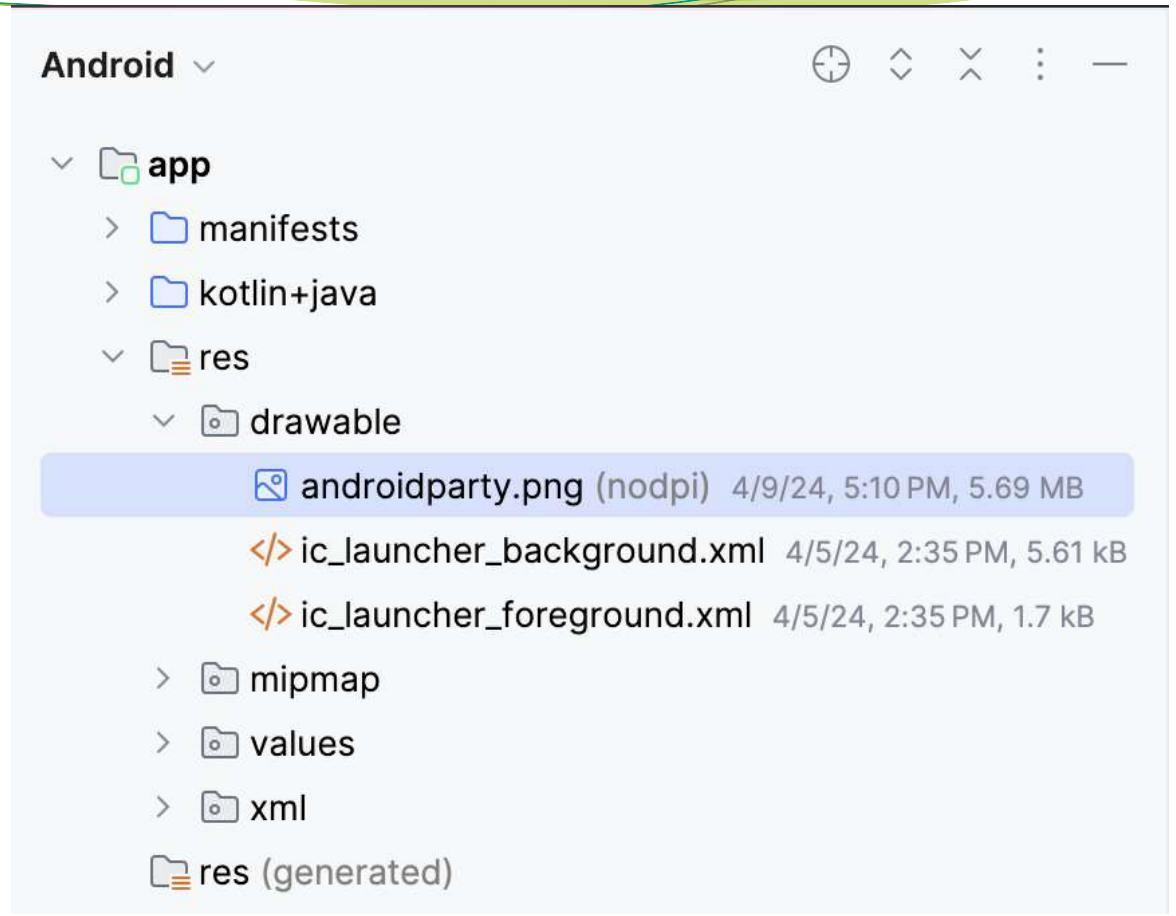


2.3 添加图片


12:00
Happy Birthday Sam!
From Emma
12:00
Happy Birthday Sam!
From Emma

2.3 添加图片

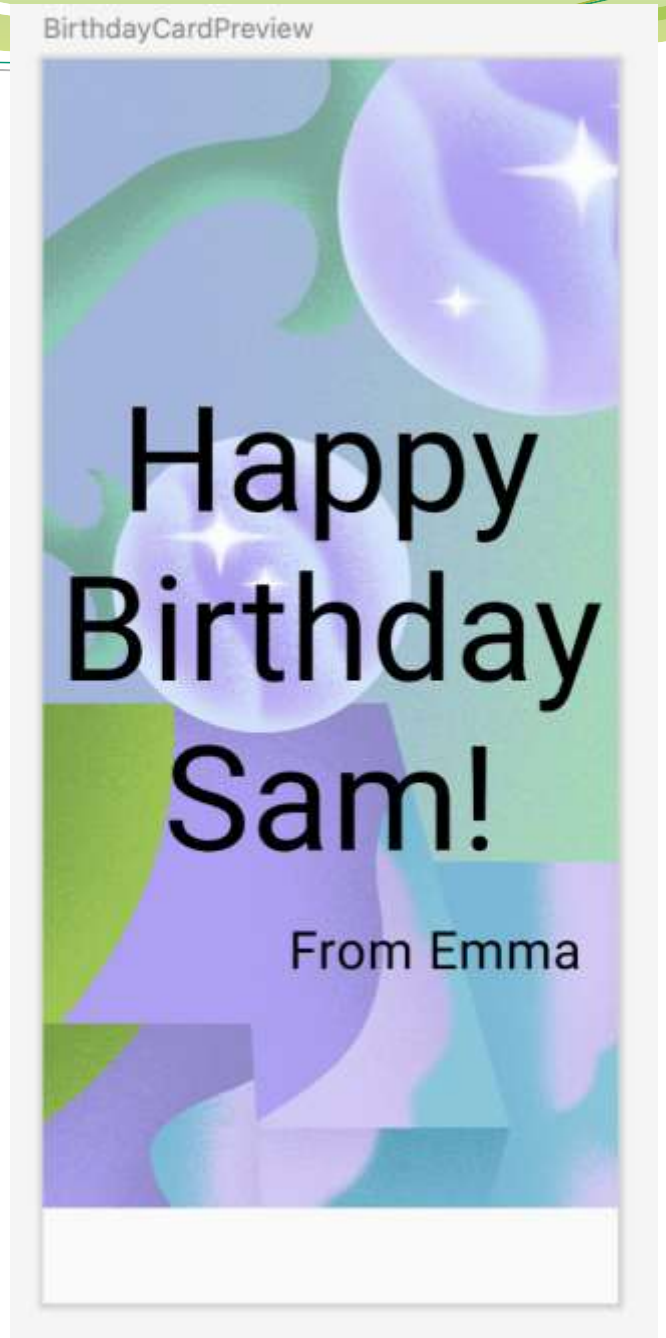




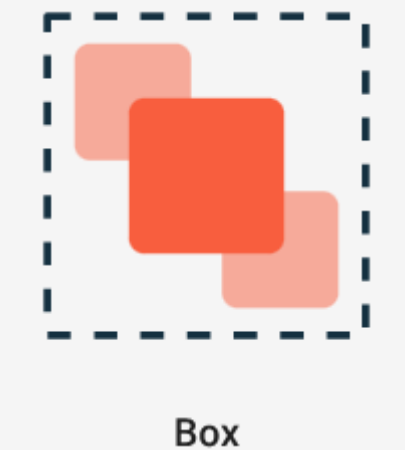

```
<resources>
    <string name="app_name">Happy Birthday</string>
    <string name="happy_birthday_text">Happy Birthday Sam!</string>
    <string name="signature_text">From Emma</string>
    <string name="title_activity_test">TestActivity</string>
</resources>
```

2.3 添加图片

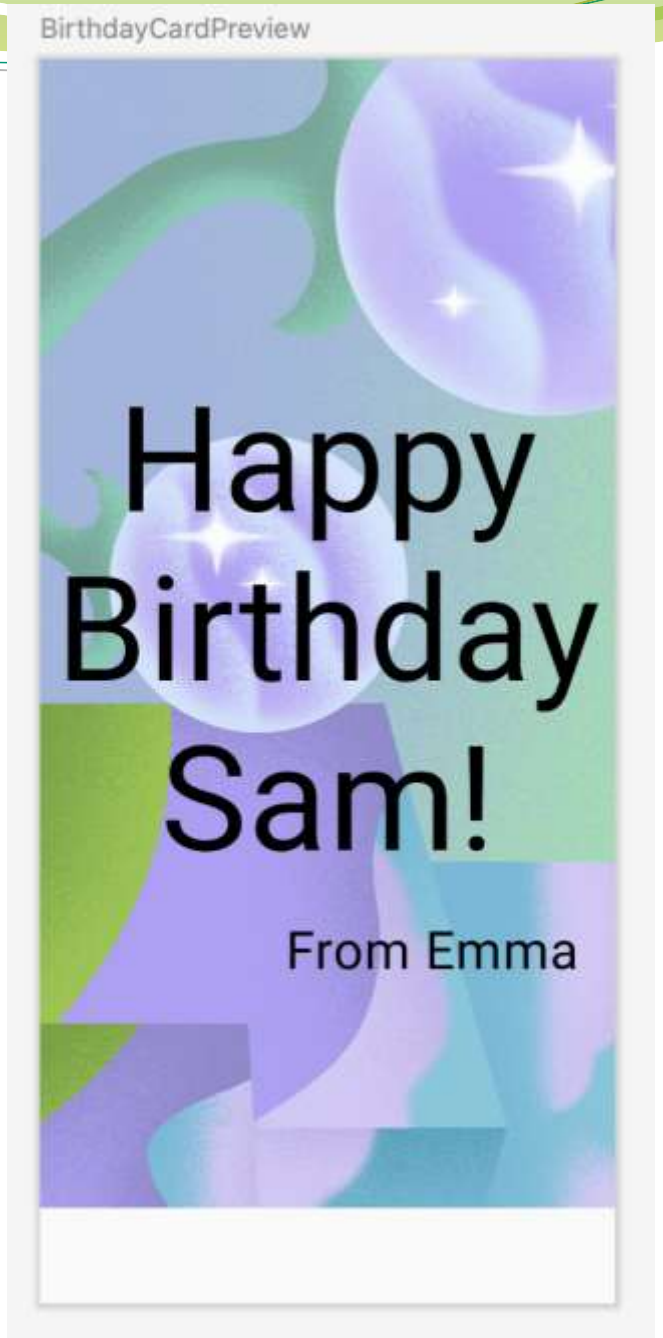
```
@Composable
fun GreetingImage(message: String, from: String) {
    val image = painterResource(R.drawable.androidparty)
    Image(
        painter = image,
        contentDescription = null
    )
    GreetingText(
        message = message,
        from = from,
        modifier = Modifier
            .fillMaxSize()
            .padding(8.dp)
    )
}

@Preview(showBackground = false)
@Composable
private fun BirthdayCardPreview() {
    HappyBirthdayTheme {
        GreetingImage(
            stringResource(R.string.happy_birthday_text),
            stringResource(R.string.signature_text)
        )
    }
}
```



2.3 添加图片

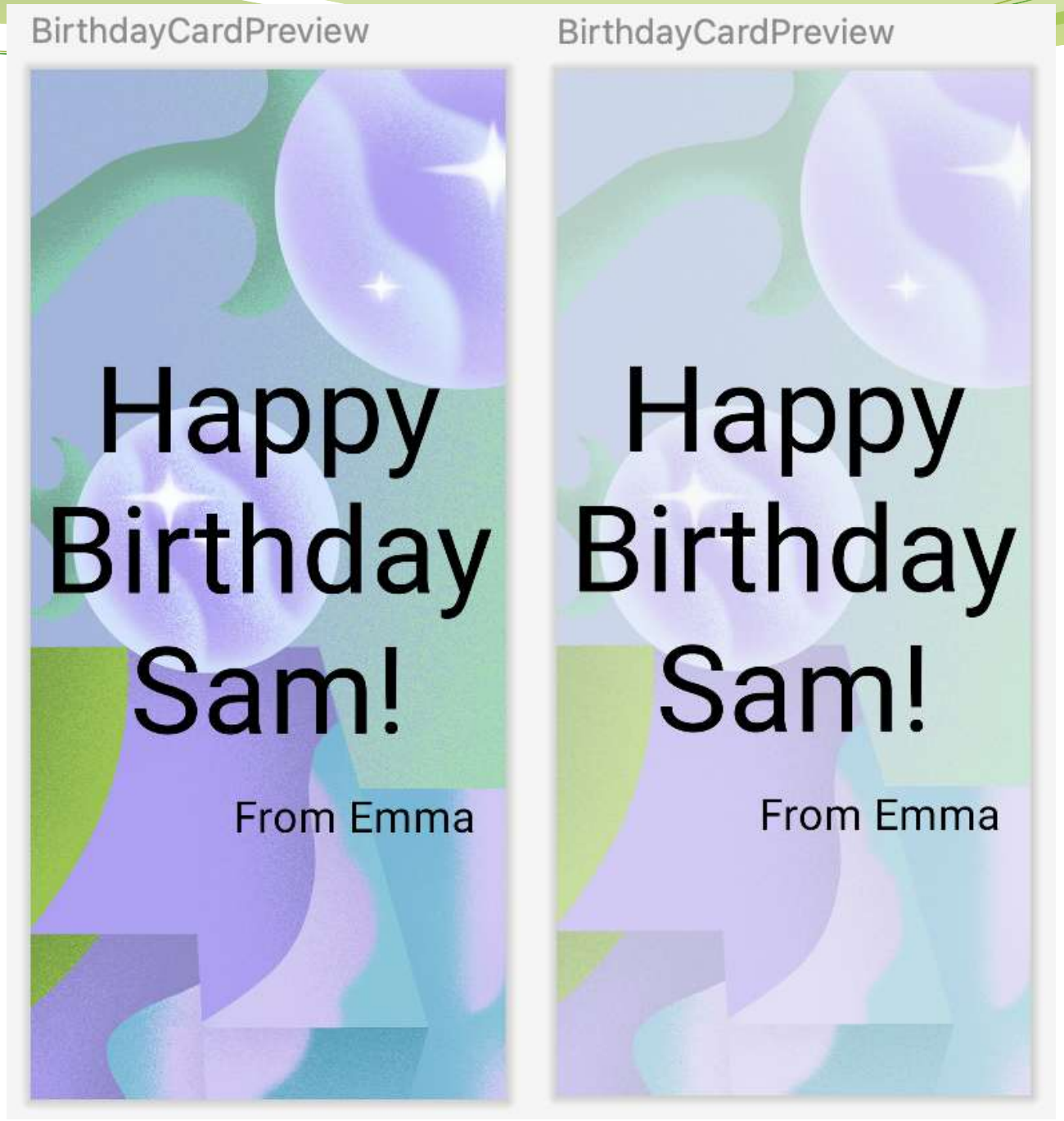



```
@Composable
fun GreetingImage(message: String, from: String, modifier: Modifier = Modifier) {
    val image = painterResource(R.drawable.androidparty)
    Box(modifier) {
        Image(
            painter = image,
            contentDescription = null
        )
        GreetingText(
            message = message,
            from = from,
            modifier = Modifier
                .fillMaxSize()
                .padding(8.dp)
        )
    }
}
```



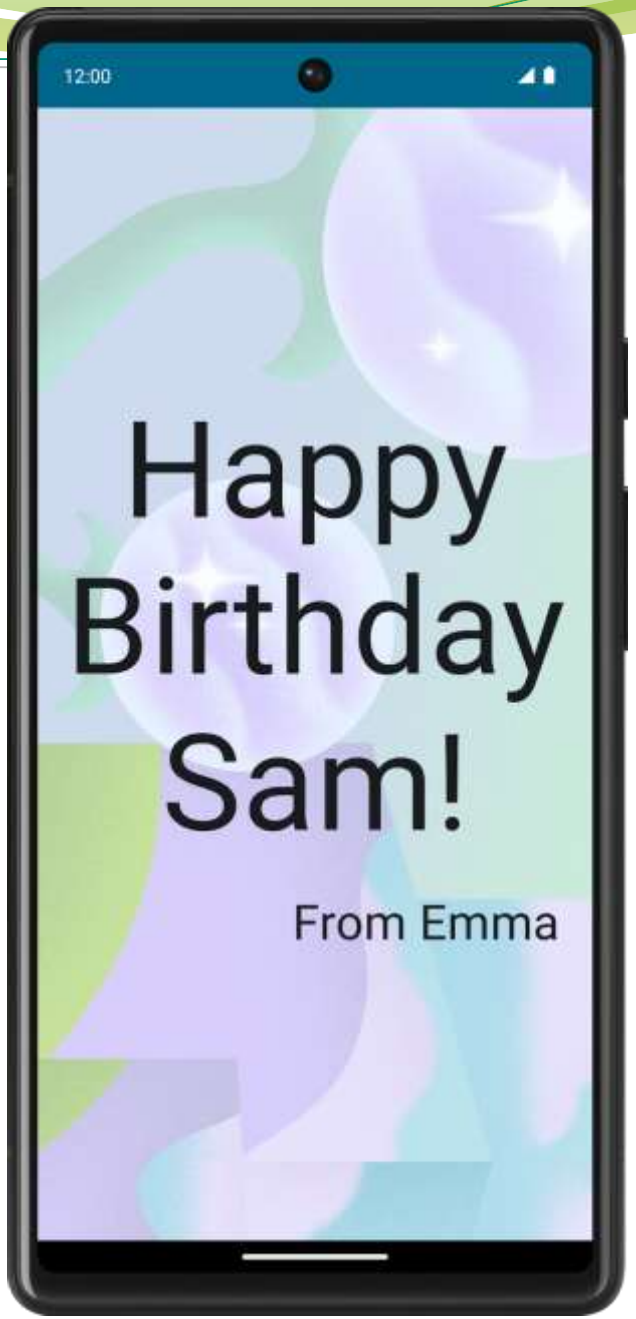
2.3 添加图片

```
Image(
    painter = image,
    contentDescription = null,
    contentScale = ContentScale.Crop,
    alpha = 0.5F
)
```

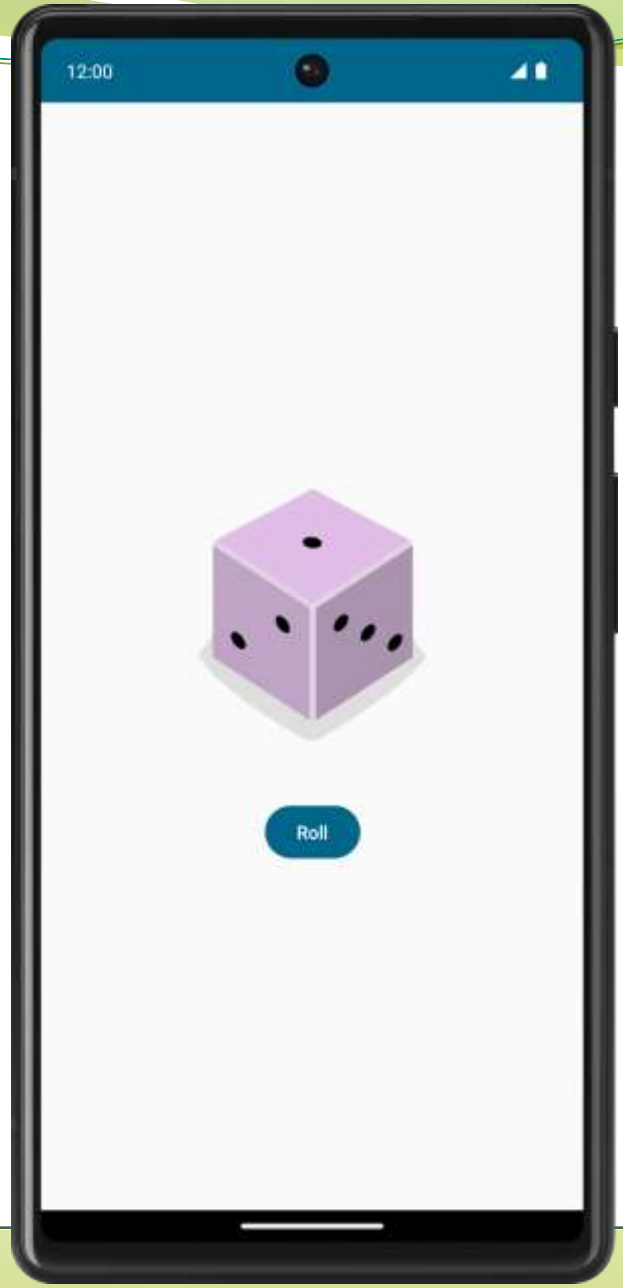


2.3 添加图片

```
setContent {
    HappyBirthdayTheme {
        // A surface container using the 'background' color from the theme
        Surface(
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background
        ) {
            GreetingImage(
                message = "Happy Birthday Sam!",
                from = "From Emma"
            )
        }
    }
}
```



3 创建交互式应用



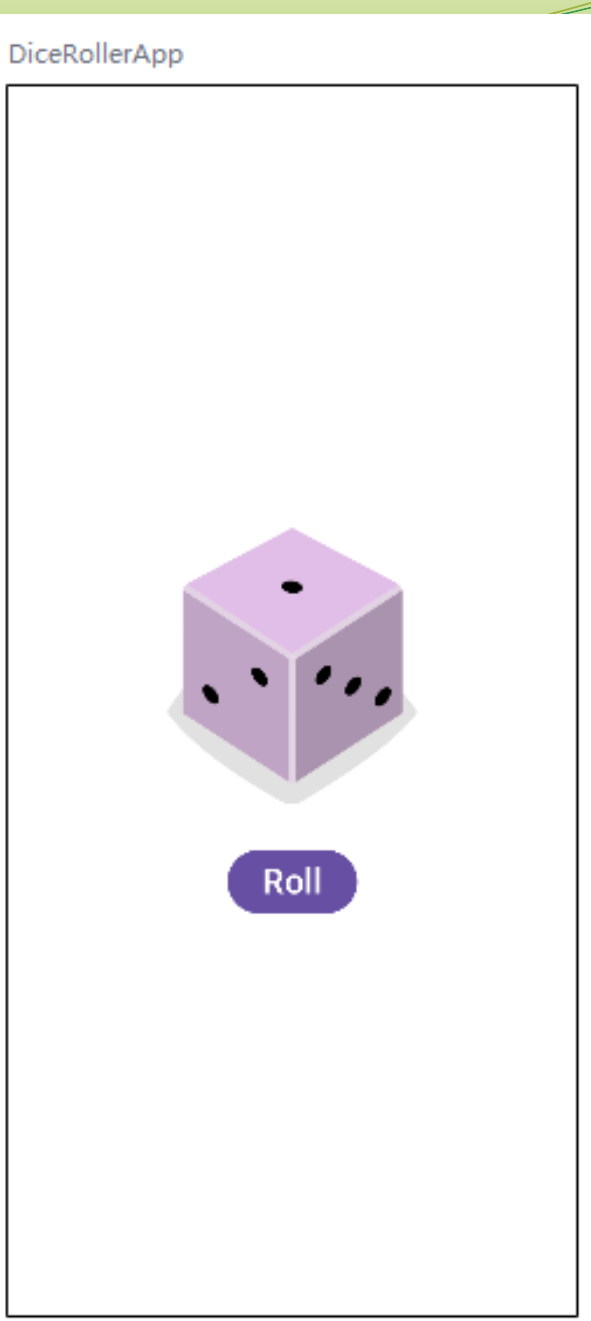
3.1 项目初始化

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            DiceRollerTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    DiceRollerApp()
                }
            }
        }
    }
}
```

```
@Preview
@Composable
fun DiceRollerApp() {
    DiceWithButtonAndImage(modifier = Modifier
        .fillMaxSize()
        .wrapContentSize(Alignment.Center)
    )
}

@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
}
```

3.2 静态设计

```
@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Image(
            painter = painterResource(R.drawable.dice_1),
            contentDescription = "1"
        )

        Button(onClick = { /*TODO*/ }) {
            Text(
                text = stringResource(R.string.roll),
                fontSize = 24.sp
            )
        }
    }
}
```

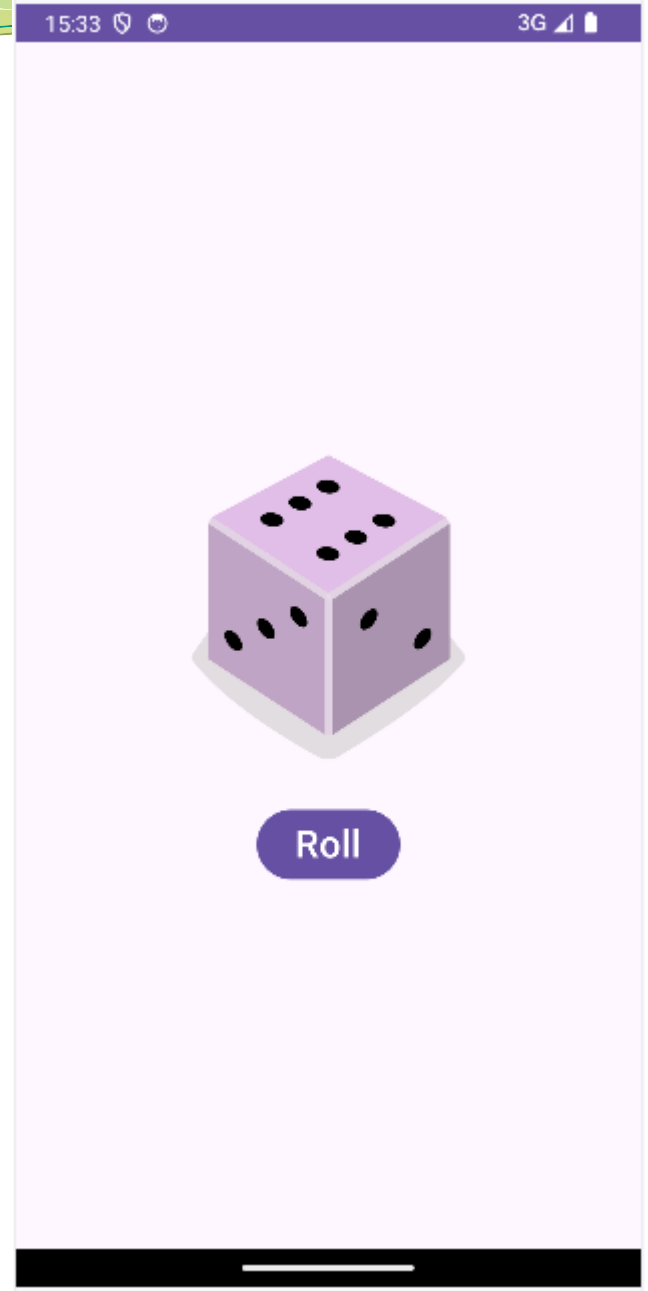


3.3 交互逻辑设计

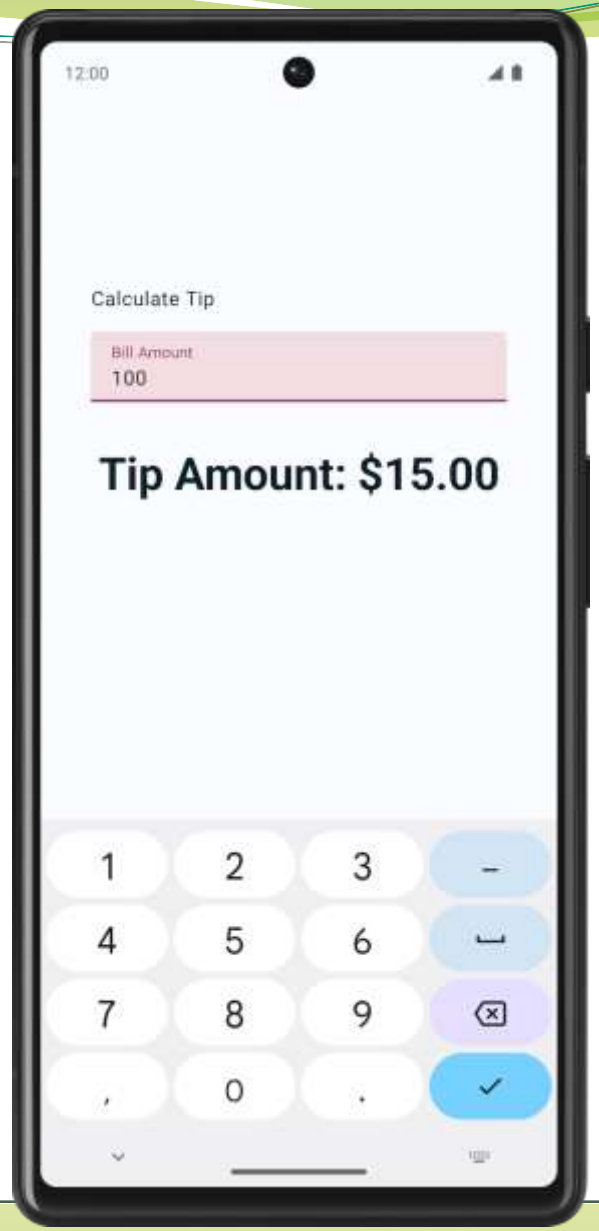
```
@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    var result = 1
    val imageResource = when(result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    Column(modifier = modifier, horizontalAlignment = Alignment.CenterHorizontally) {
        Image(painter = painterResource(imageResource), contentDescription = result.toString())

        Button(
            onClick = { result = (1..6).random() },
        ) {
            Text(text = stringResource(R.string.roll), fontSize = 24.sp)
        }
    }
}
```

3.3 交互逻辑设计

```
@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    var result by remember { mutableIntStateOf( value: 1) }
    val imageResource = when(result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    Column(modifier = modifier, horizontalAlignment = Alignment.CenterHorizontally) {
        Image(painter = painterResource(imageResource), contentDescription = result.toString())

        Button(
            onClick = { result = (1 ≤ .. ≤ 6).random() },
        ) {
            Text(text = stringResource(R.string.roll), fontSize = 24.sp)
        }
    }
}
```

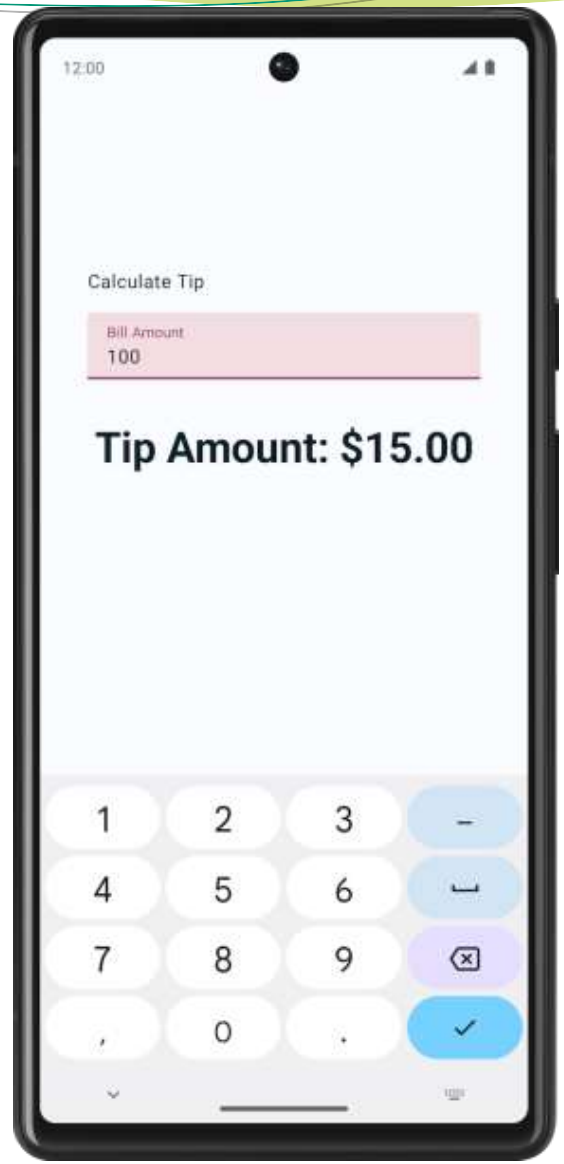


4 与状态交互


12:00
Calculate Tip
Bill Amount
100
Tip Amount: $15.00
1
2
3
4
5
6
7
8
9
,
0
.

4.1 Compose 中的状态

- State状态
  - 任何可能随时间变化并且会影响 UI 的数据。
  - 当状态发生变化时，Compose会自动重新执行受影响的组件以更新界面。



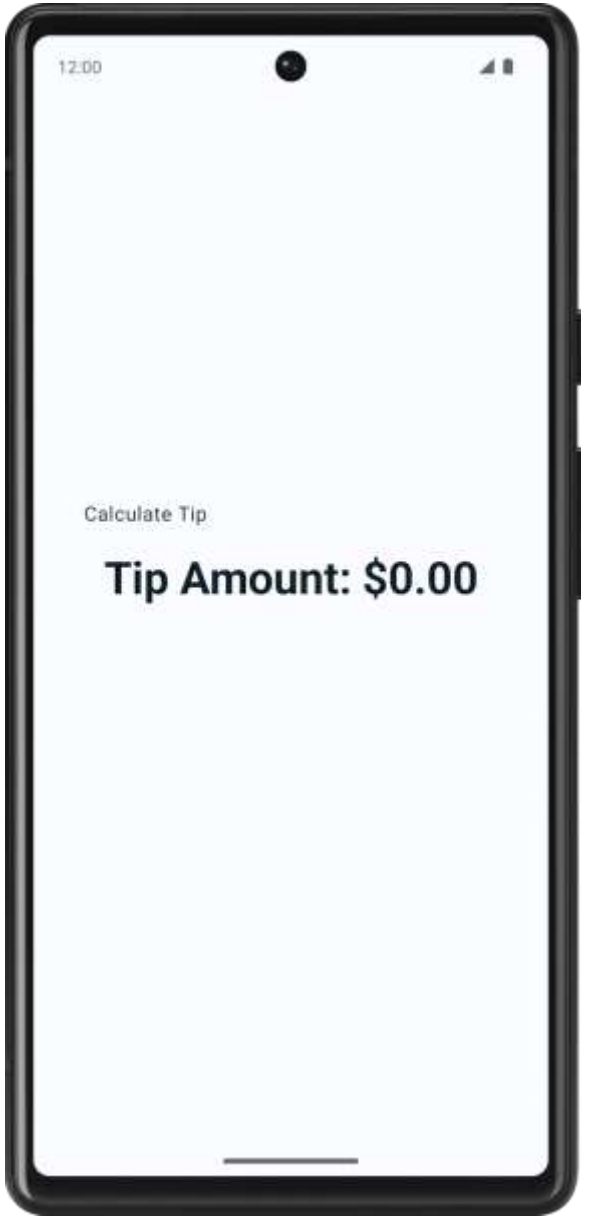
4.2 初始界面设计

```
private fun calculateTip(amount: Double, tipPercent: Double = 15.0): String {
    val tip = tipPercent / 100 * amount
    return NumberFormat.getCurrencyInstance().format(tip)
}
```

```
@Composable
fun TipTimeLayout() {
    Column(
        modifier = Modifier
            .statusBarsPadding()
            .padding(horizontal = 40.dp)
            .verticalScroll(rememberScrollState())
            .safeDrawingPadding(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = stringResource(R.string.calculate_tip),
            modifier = Modifier
                .padding(bottom = 16.dp, top = 40.dp)
                .align(alignment = Alignment.Start)
        )
        Text(
            text = stringResource(R.string.tip_amount, "$0.00"),
            style = MaterialTheme.typography.displaySmall
        )
        Spacer(modifier = Modifier.height(150.dp))
    }
}
```

```
override fun onCreate(savedInstanceState: Bundle?) {
    //...
    setContent {
        TipTimeTheme {
            Surface(
            //...
            ) {
                TipTimeLayout()
            }
        }
    }
}
```

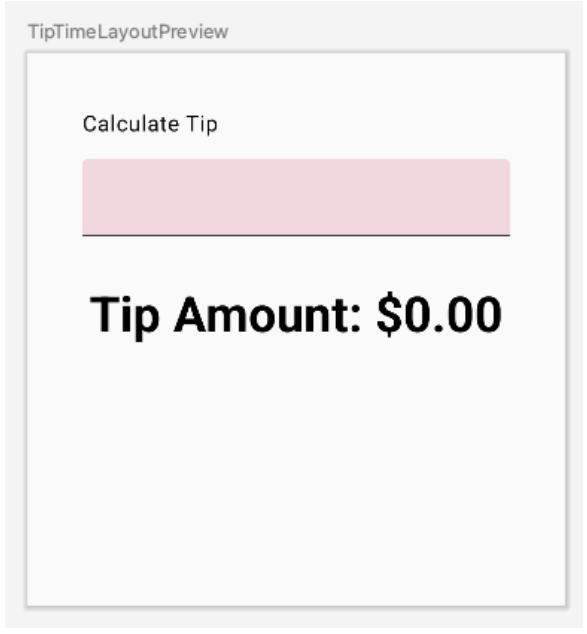
```
@Preview(showBackground = true)
@Composable
fun TipTimeLayoutPreview() {
    TipTimeTheme {
        TipTimeLayout()
    }
}
```



4.3 接收用户输入


```
@Composable
fun EditNumberField(modifier: Modifier = Modifier) {
    TextField(
        value = "",
        onValueChange = {},
        modifier = modifier
    )
}
```

```
@Composable
fun TipTimeLayout() {
    Column(
        modifier = Modifier
            .statusBarsPadding()
            .padding(horizontal = 40.dp)
            .verticalScroll(rememberScrollState())
            .safeDrawingPadding(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            ...
        )
        EditNumberField(modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth())
        Text(
            ...
        )
        ...
    }
}
```



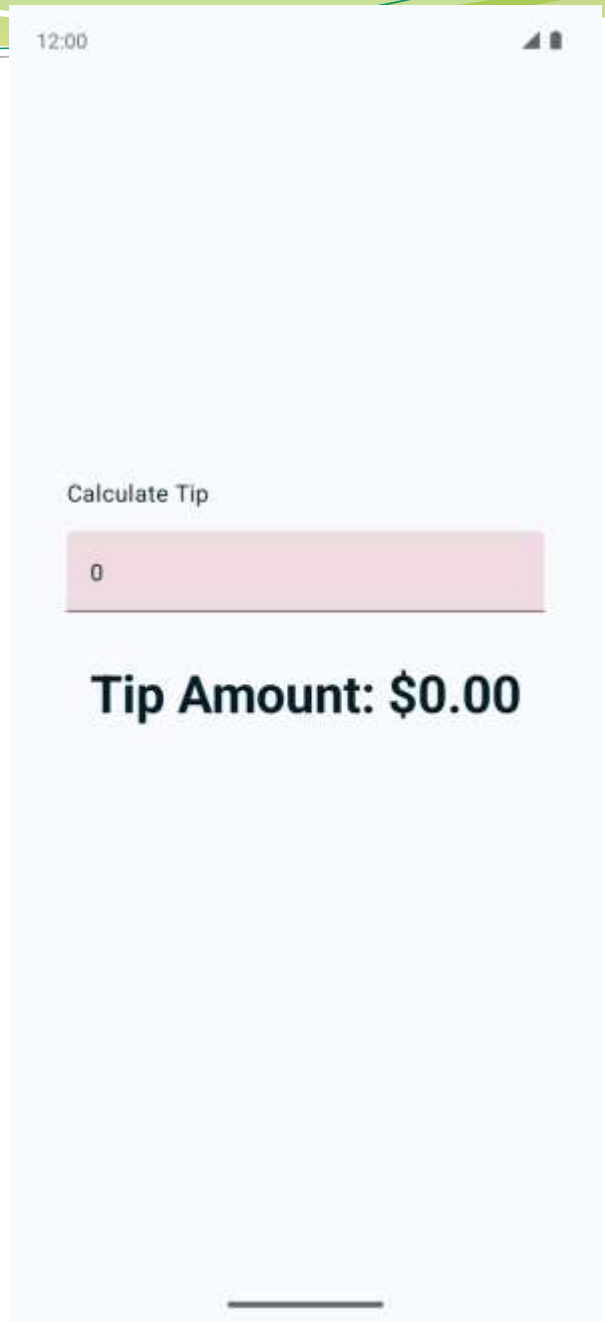
4.3 接收用户输入

```
val amountInput = "0"
TextField(
    value = amountInput,
    onValueChange = {},
)
```

```
var amountInput: MutableState<String> = mutableStateOf("0")
```

```
var amountInput = mutableStateOf("0")
```

```
@Composable
fun EditNumberField(modifier: Modifier = Modifier) {
    var amountInput = mutableStateOf("0")
    TextField(
        value = amountInput.value,
        onValueChange = { amountInput.value = it },
        modifier = modifier
    )
}
```

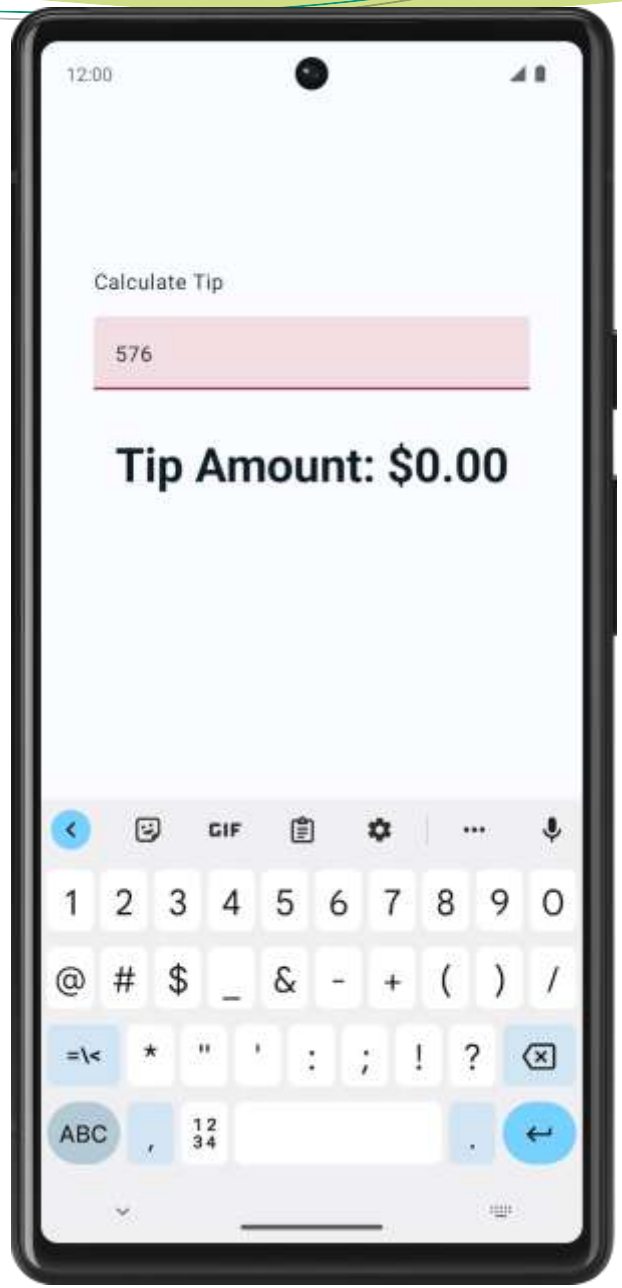


4.3 接收用户输入

```
var amountInput by remember { mutableStateOf("") }
```

```
val amountInputState = remember { mutableStateOf("") }
var amountInput: String
    get() = amountInputState.value
    set(value) {
        amountInputState.value = value
    }
```

```
@Composable
fun EditNumberField(modifier: Modifier = Modifier) {
    var amountInput by remember { mutableStateOf("") }
    TextField(
        value = amountInput,
        onValueChange = { amountInput = it },
        modifier = modifier
    )
}
```

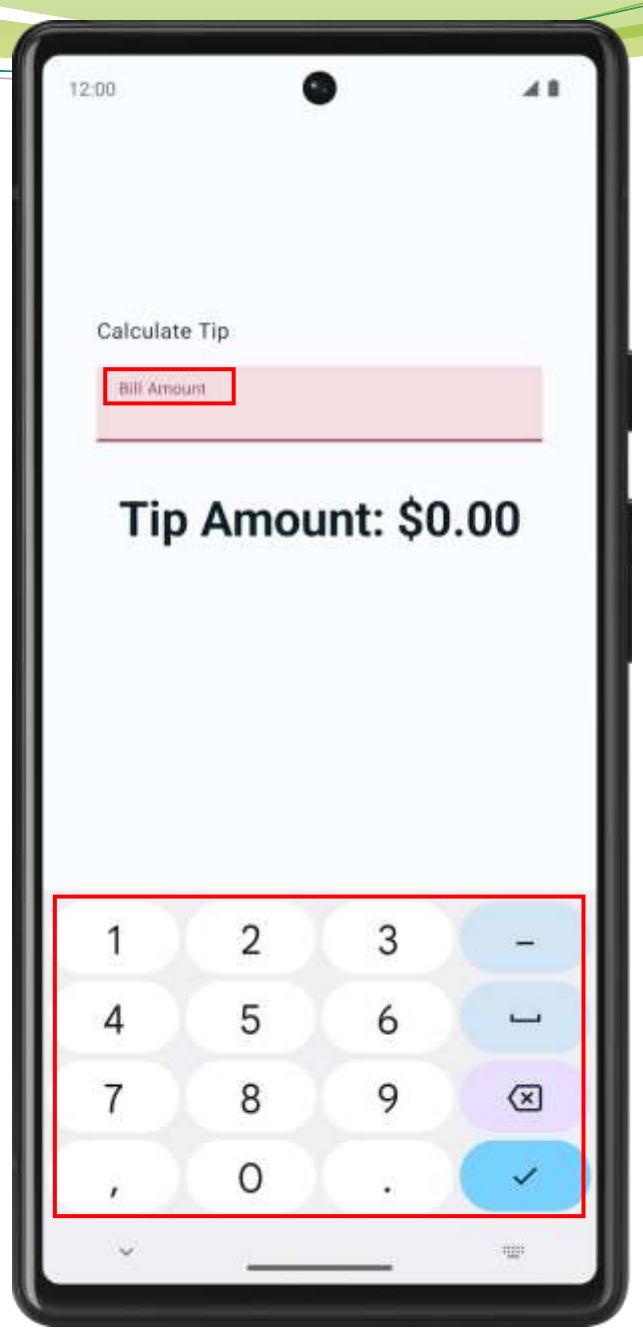


4.4 优化外观

① Label

② Label
Input text

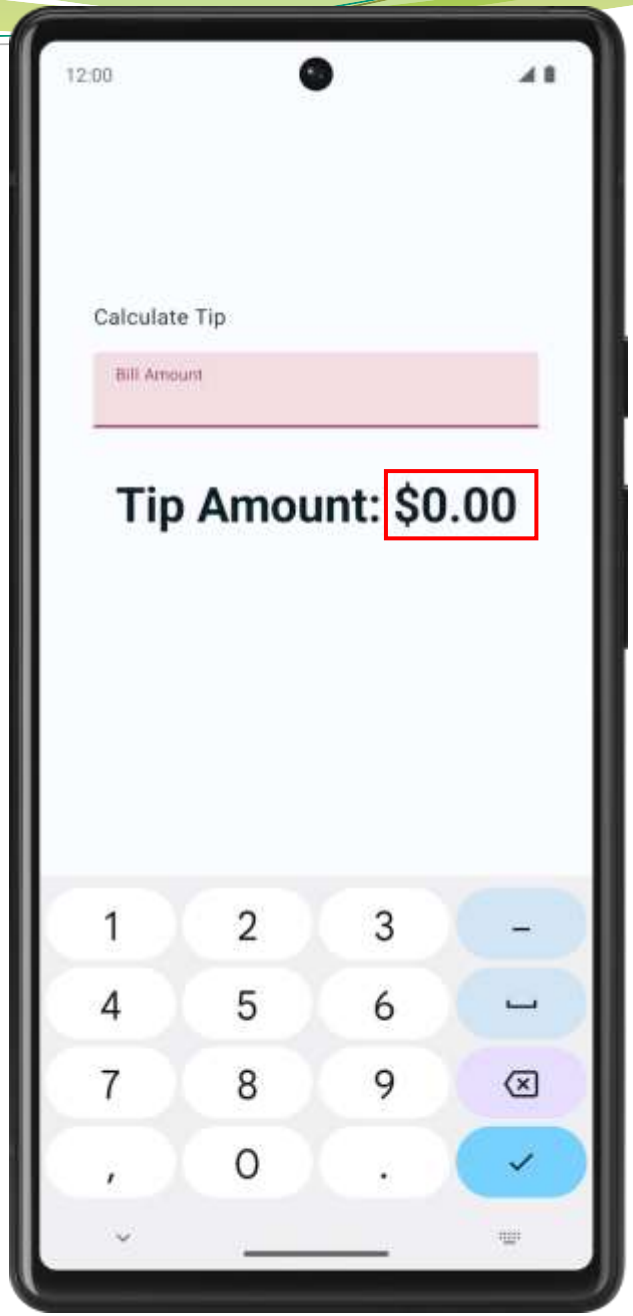
```
@Composable
fun EditNumberField(modifier: Modifier = Modifier) {
    var amountInput by remember { mutableStateOf("") }
    TextField(
        value = amountInput,
        onValueChange = { amountInput = it },
        singleLine = true,
        label = { Text(stringResource(R.string.bill_amount)) },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        modifier = modifier
    )
}
```



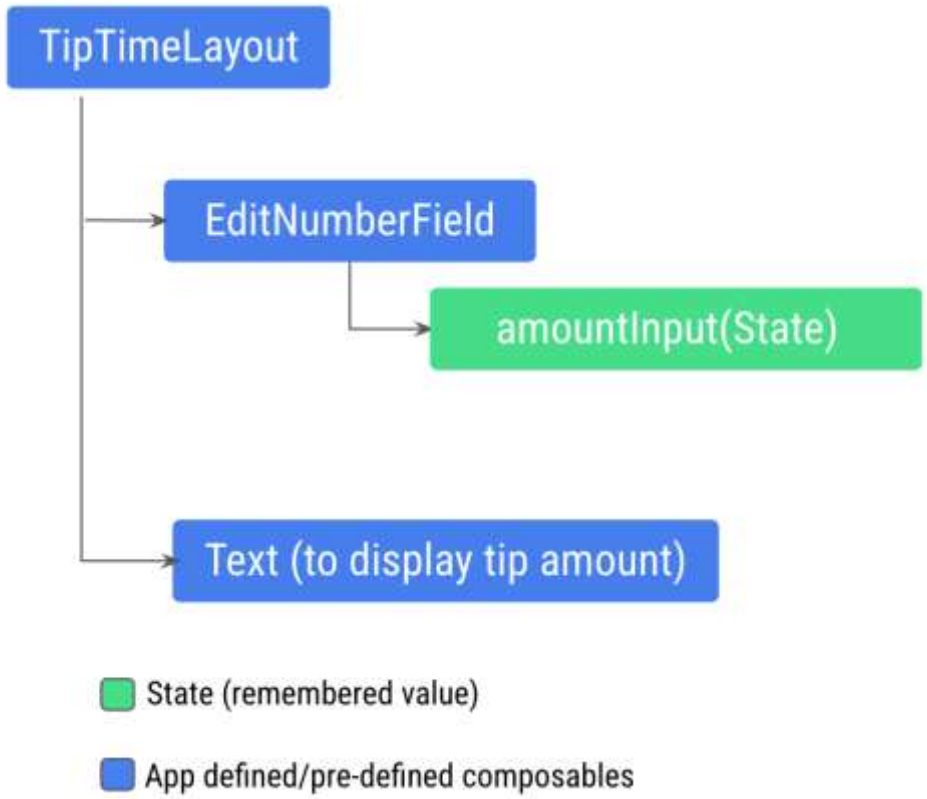
4.5 状态提升

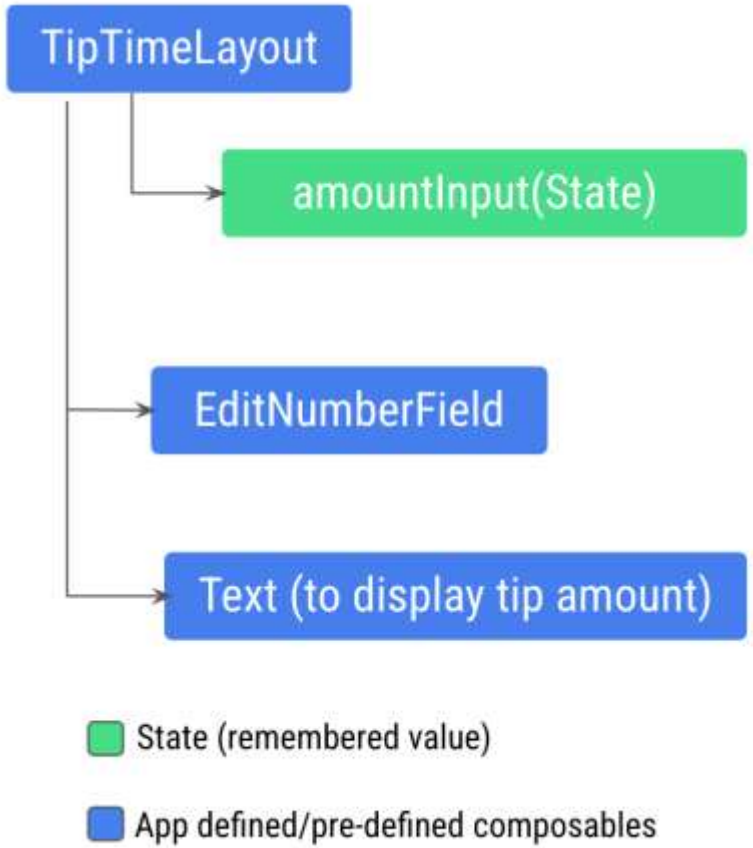
```
private fun calculateTip(amount: Double, tipPercent: Double = 15.0): String {
    val tip = tipPercent / 100 * amount
    return NumberFormat.getCurrencyInstance().format(tip)
}
```

```
@Composable
fun EditNumberField(modifier: Modifier = Modifier) {
    var amountInput by remember { mutableStateOf("") }

    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount)

    TextField(
        value = amountInput,
        onValueChange = { amountInput = it },
        label = { Text(stringResource(R.string.bill_amount)) },
        modifier = Modifier.fillMaxWidth(),
        singleLine = true,
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number)
    )
}
```

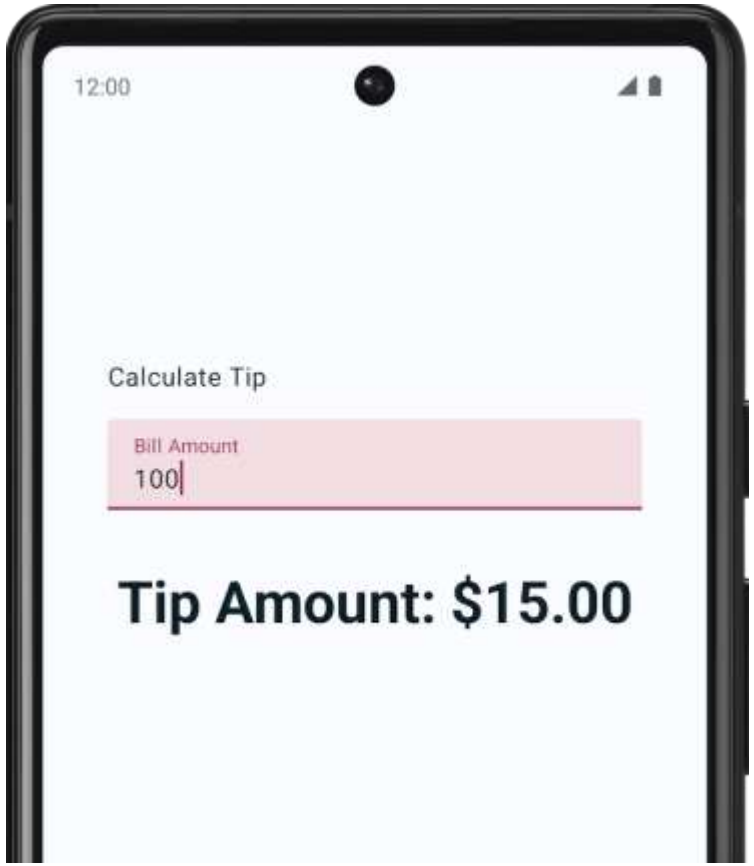


4.5 状态提升


TipTimeLayout
EditNumberField
amountInput(State)
Text (to display tip amount)
State (remembered value)
App defined/pre-defined composables

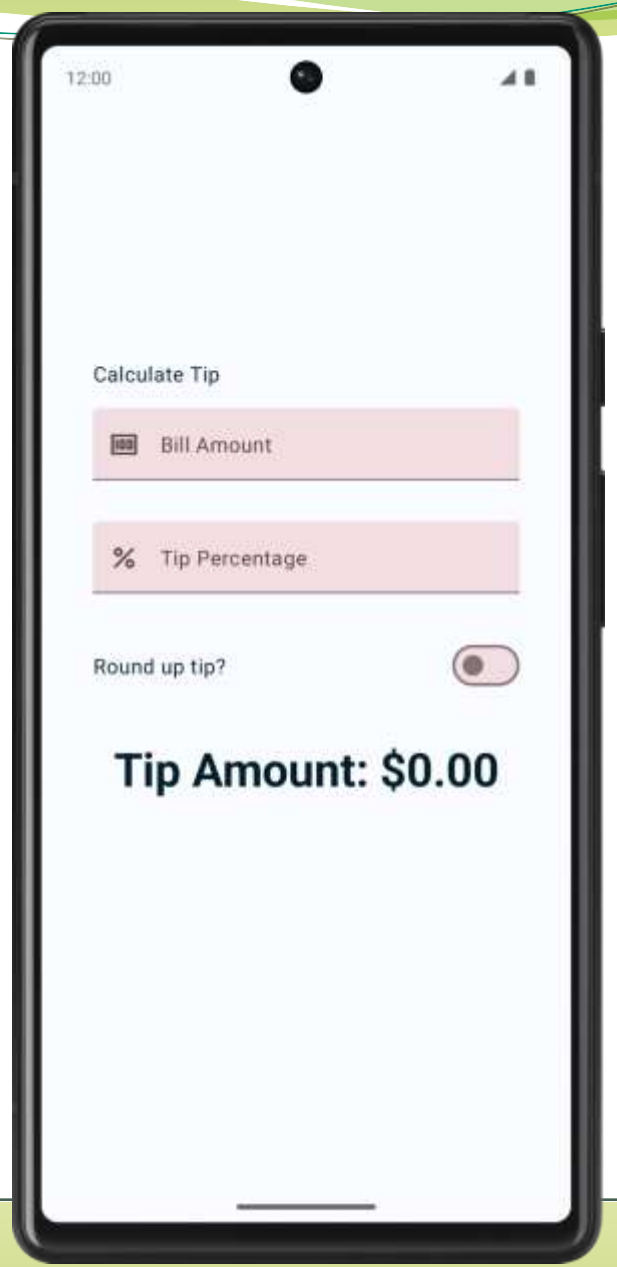

TipTimeLayout
amountInput(State)
EditNumberField
Text (to display tip amount)
State (remembered value)
App defined/pre-defined composables

4.5 状态提升

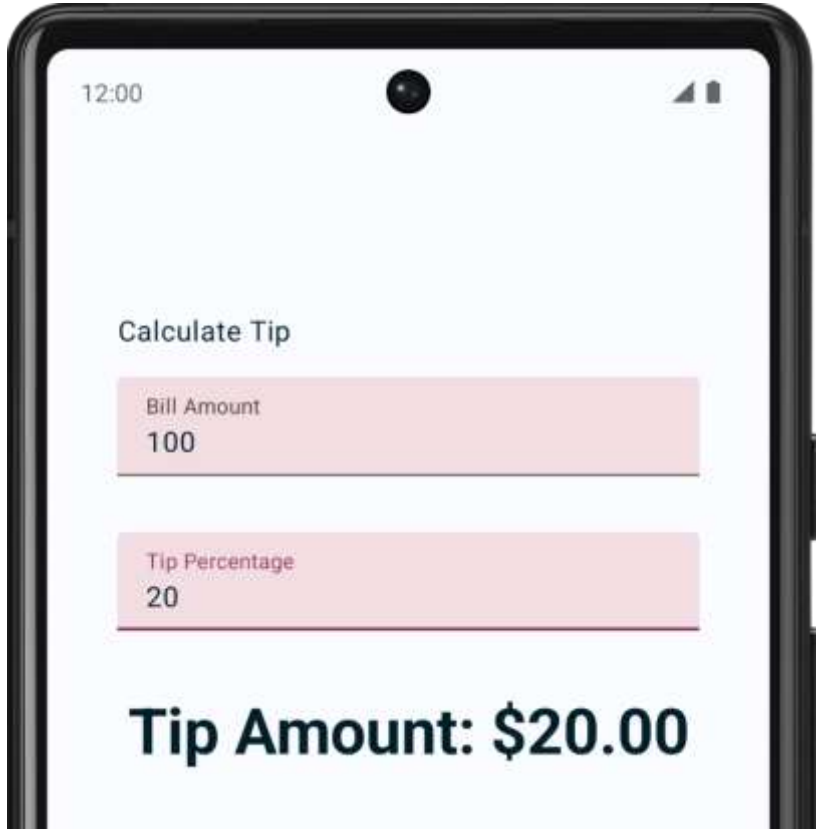



```
@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf("") }
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount)

    Column(......) {
        Text(......)
        EditNumberField(
            value = amountInput,
            onValueChange = { amountInput = it },
            modifier = ......
        )
        Text(
            text = stringResource(R.string.tip_amount, tip),
            ......
        )
        ......
    }
}

@Composable
fun EditNumberField(
    value: String,
    onValueChange: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        value = value,
        onValueChange = onValueChange,
        ......
    )
}
```

5 UI优化

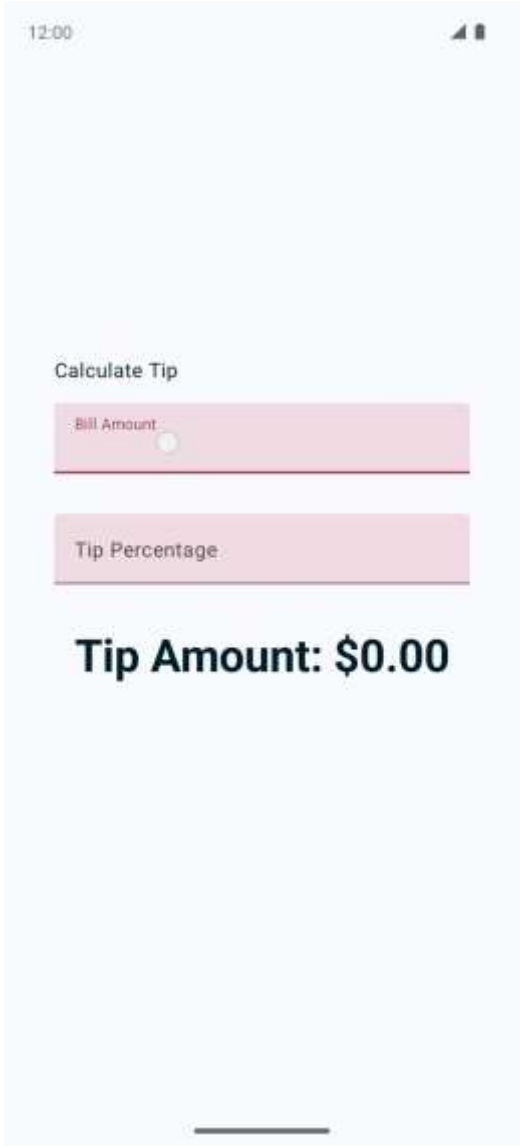

12:00
Calculate Tip
Bill Amount
Tip Percentage
Round up tip?
Tip Amount: $0.00

5.1 自定义小费百分比



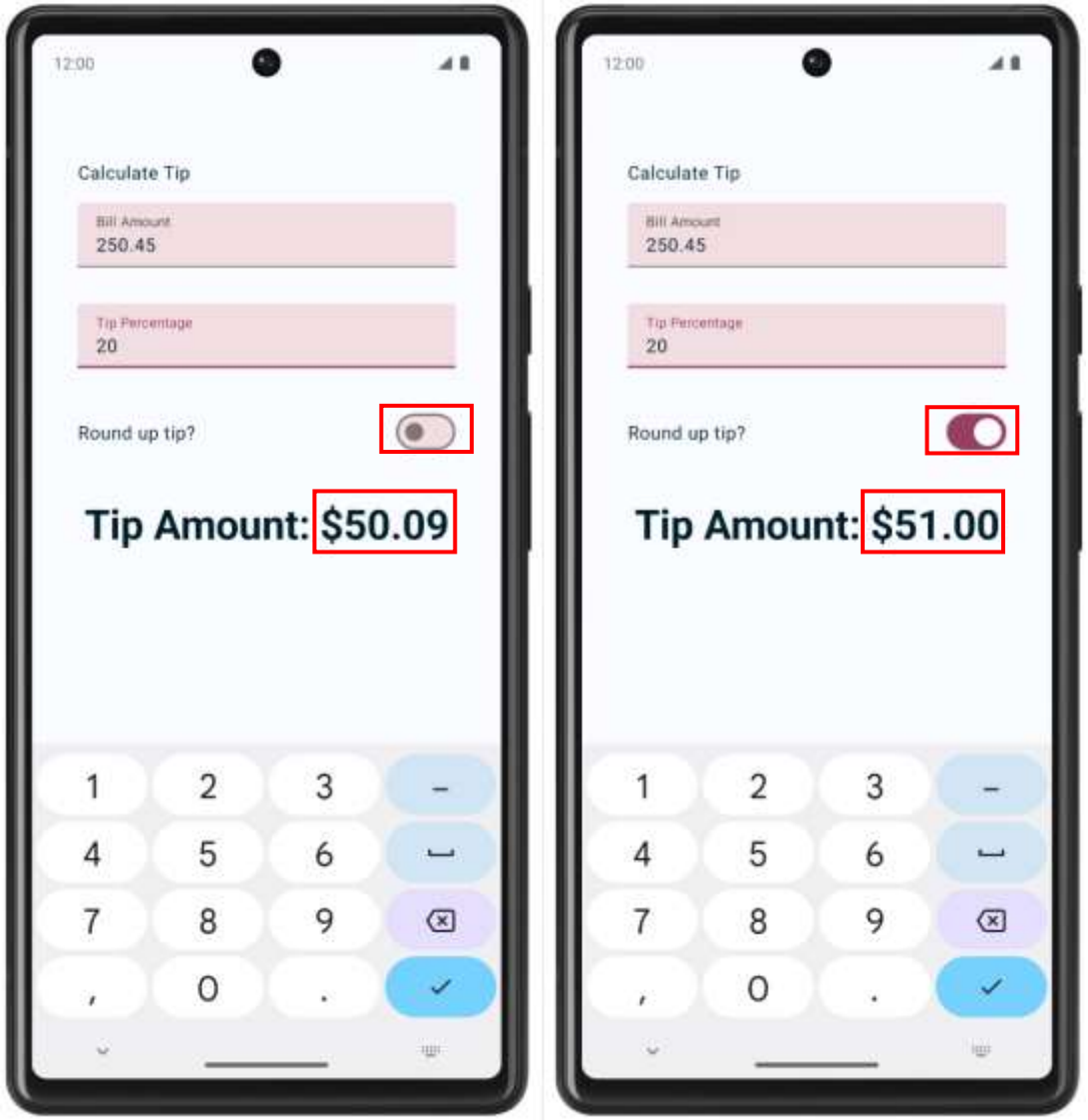

```
@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf("") }
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    var tipInput by remember { mutableStateOf("") }
    val tipPercent = tipInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount, tipPercent)

    Column(......) {
        ......
        EditNumberField(
            label = R.string.bill_amount,
            value = amountInput,
            onValueChanged = { amountInput = it },
            ......
        )
        EditNumberField(
            label = R.string.how_was_the_service,
            value = tipInput,
            onValueChanged = { tipInput = it },
            ......
        )
        ......
    }
}

@Composable
fun EditNumberField(
    @StringRes label: Int,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        label = { Text(stringResource(label)) },
        ......
    )
}
```

5.2 设置键盘操作按钮




```
@Composable
fun TipTimeLayout() {
    ......
    Column(......) {
        ......
        EditNumberField(
            label = R.string.bill_amount,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Next
            ),
            ......
        )
        EditNumberField(
            label = R.string.bill_amount,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Done
            ),
            ......
        )
        ......
    }
}

@Composable
fun EditNumberField(
    @StringRes label: Int,
    keyboardOptions: KeyboardOptions,
    ......
){
    TextField(
        ......
        keyboardOptions = keyboardOptions
    )
}
```

5.3 添加开关




```
@Composable
fun TipTimeLayout() {
    ......
    var roundUp by remember { mutableStateOf(false) }
    val tip = calculateTip(amount, tipPercent, roundUp)

    Column(......) {
        ......
        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )
        ......
    }
}
```

```
@Composable
fun RoundTheTipRow(
    roundUp: Boolean,
    onRoundUpChanged: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    Row(
        modifier = modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = stringResource(R.string.round_up_tip))
        Switch(
            modifier = Modifier
                .fillMaxWidth()
                .wrapContentWidth(Alignment.End),
            checked = roundUp,
            onCheckedChange = onRoundUpChanged
        )
    }
}
```

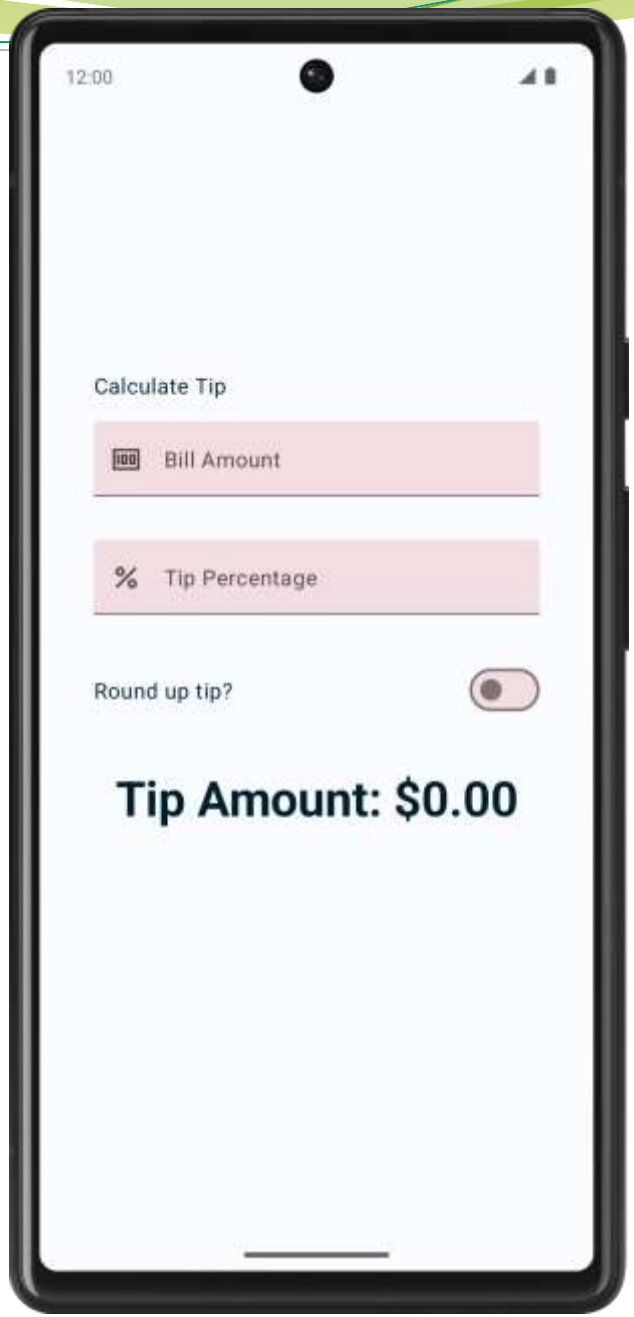
```
private fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return NumberFormat.getCurrencyInstance().format(tip)
}
```

5.4 支持横屏显示

```
@Composable
fun TipTimeLayout() {
    // ...
    Column(
        modifier = Modifier
            .padding(40.dp)
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        //...
    }
}
```

5.5 文本框前置图标

```
@Composable
fun TipTimeLayout() {
    ......
    Column(......) {
        ......
        EditNumberField(
            ......
            leadingIcon = R.drawable.money,
            ......
        )
        EditNumberField(
            ......
            leadingIcon = R.drawable.percent,
            ......
        )
        ......
    }
}

@Composable
fun EditNumberField(
    ......
    @DrawableRes leadingIcon: Int,
    .......
) {
    TextField(
        ......
        leadingIcon = { Icon(painter = painterResource(id = leadingIcon), null) },
        ......
    )
}
```



感谢观看！